

PROGRAMMAZIONE II

COSTRUTTORI DI CLASSE E INCAPSULAMENTO

Michele Pasqua, PhD



UNIVERSITÀ
di **VERONA**
Dipartimento
di **INFORMATICA**

A.A. 2025/2026

COSTRUTTORI DI CLASSE



COSTRUTTORE DI CLASSE

Per creare un oggetto `Clazz` in memoria usiamo `Clazz myClazz = new Clazz();`

Cosa accade dietro le quinte

- ▶ Viene allocata nell'heap memoria sufficiente a contenere un oggetto `Clazz`
- ▶ Un riferimento (che punta alla memoria appena allocata) viene fornito in `myClazz`
- ▶ Tutti i campi dell'oggetto puntato da `myClazz` vengono inizializzati
- ▶ Il costruttore di `Clazz` viene chiamato sull'oggetto puntato da `myClazz`

Costruttore DI CLASSE (CONT.)

Un costruttore di classe è un metodo speciale

- ▶ Che ha lo stesso nome della classe
- ▶ Che non ha alcun valore di ritorno (neanche `void`)
- ▶ Che viene eseguito solo alla creazione dell'istanza di classe
- ▶ Viene usato per fornire un'inizializzazione personalizzata ai campi della classe

Infatti, un costruttore può avere zero, uno o più parametri

Costruttore Parametrizzato

Un costruttore parametrizzato è un costruttore che ha parametri

```
public class Car {                                     1
    String color;                                     2
                                                    3
    Car(String color) { // costruttore parametrizzato 4
        paint(color); // i metodi possono essere chiamati dai costruttori 5
    }                                                  6
                                                    7
    void paint(String color) {                         8
        this.color = color;                          9
    }                                                  10
}                                                      11
```

Possiamo creare un oggetto Car con `Car myCar = new Car("yellow");`

COSTRUTTORE DI DEFAULT

Se non viene fornito alcun costruttore, il compilatore aggiunge un costruttore di default

```
public class Car {  
    String color;  
  
    Car() { } // costruttore di default  
  
    void paint(String color) {  
        this.color = color;  
    }  
}
```

1
2
3
4
5
6
7
8
9

Questo è il motivo per cui possiamo creare un oggetto Car con `Car myCar = new Car();`

FALSO COSTRUTTORE

Se viene fornito un valore di ritorno abbiamo un falso costruttore

```
public class Car {                                1
    String color;                                  2

    void Car(String color) { // il costruttore di default viene aggiunto ed 3
        this.color = color; // eseguito al posto di questo metodo          4
    }                                              5
                                                6
    void paint(String color) {                      7
        this.color = color;                        8
    }                                              9
}                                                  10
                                                11
```

`void Car(String color)` è visto come un normale metodo (non chiamato all'istanziamento)

COSTRUTTORE DI COPIA

Un costruttore di copia inizializza un oggetto usando un altro oggetto della stessa classe

- ▶ Java non fornisce costruttori di copia, i programmatori dovrebbero fornirli

```
1 public class Car {
2     String color;
3
4     Car(Car carObj) { // costruttore di copia
5         this.color = carObj.color;
6     }
7
8     void paint(String color) {
9         this.color = color;
10    }
11 }
```


OVERLOADING DEI COSTRUTTORI

Una classe può essere inizializzata in più modi, sfruttando l'overloading

```
public class Car {                                1
    String color;                                  2

    Car(String color) {    // costruttore parametrizzato  3
        this.color = color;                                4
    }                                                        5
                                                        6
    Car(Car carObj) {    // costruttore di copia          7
        this.color = carObj.color;                        8
    }                                                        9
                                                        10
    void paint(String color) { this.color = color; }      11
}                                                         12
                                                         13
```

Possiamo fare `Car myCar1 = new Car("red");` e `Car myCar2 = new Car(myCar1);`

OVERLOADING DEI COSTRUTTORI (CONT.)

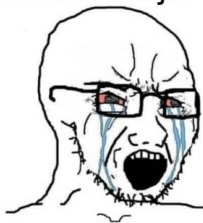
Con `this` possiamo anche fare riferimento a un altro costruttore (della stessa classe)

```
public class Car {                                     1
    String color;                                     2

    Car(String color) { // costruttore parametrizzato 3
        this.color = color;                           4
    }                                                   5
                                                    6
    Car(Car carObj) { // costruttore di copia          7
        this(carObj.color); // chiama il costruttore parametrizzato 8
    }                                                   9
                                                    10
    void paint(String color) { this.color = color; } 11
}                                                       12
                                                    13
```

OVERLOADING

Math major:



**abuse
of notation**

Programmer:



**operator
overloading**

DISTRUTTORE DI OGGETTI

In Java, i programmatori non devono preoccuparsi della deallocazione della memoria

- ▶ Ma il linguaggio consente di eseguire codice durante la garbage collection
- ▶ Implementando un metodo `finalize()` personalizzato // deprecato da Java 9

```
public class Car {  
    String color;  
  
    void paint(String color) { this.color = color; }  
  
    protected void finalize() { // non usare i finalizer!  
        System.out.println("Farewell_" + this);  
    }  
}
```

1
2
3
4
5
6
7
8
9

INCAPSULAMENTO



INCAPSULAMENTO

Incapsulamento: raggruppare dati (campi) e operazioni (metodi) che agiscono su quei dati in una singola unità (classe)

```
public class Person { 1
    String name; 2
    int age = -1; 3

    Person(String name) { this.name = name; } 4
    5
    void changeAge(int age) { 6
        if (age > 0 && age < 122) this.age = age; 7
        else System.out.println("Invalid age."); 8
    } 9
} 10
11
```

Ok, ma possiamo ancora fare `Person sam = new Person("Sam"); sam.age = -2;`

INFORMATION HIDING

Information hiding: limitare l'accesso diretto ai dati e ai dettagli di implementazione di un oggetto dal mondo esterno

Possiamo ottenerlo combinando l'incapsulamento con i modificatori di accesso

- ▶ L'incapsulamento fornisce modularità
- ▶ I modificatori di accesso limitano l'accesso ai dati

Alcuni dati sono accessibili solo dalle istanze della classe (**private**), mentre altri possono essere accessibili anche dal mondo esterno (**public**)

INFORMATION HIDING (CONT.)

Information hiding: limitare l'accesso diretto ai dati e ai dettagli di implementazione di un oggetto dal mondo esterno

```
public class Person { 1
    String name; // la visibilità di default è public 2
    private int age = -1; 3
    4
    Person(String name) { this.name = name; } 5
    6
    void changeAge(int age) { 7
        if (age > 0 && age <= 122) this.age = age; 8
        else System.out.println("Invalid age."); 9
    } 10
} 11
```

Ora `Person sam = new Person("Sam"); sam.age = -2;` genera un errore di compilazione

INFORMATION HIDING (CONT.)

Information hiding: limitare l'accesso diretto ai dati e ai dettagli di implementazione di un oggetto dal mondo esterno

Consente di

- ▶ Scrivere codice modulare e riutilizzabile
- ▶ Garantire l'integrità dei dati interni
- ▶ Garantire la riservatezza dei dettagli di implementazione
- ▶ Facilitare l'evoluzione del codice

Programmazione a interfacce: per usare un componente basta l'interfaccia esposta, dati interni e implementazione possono cambiare liberamente

INFORMATION HIDING (CONT.)

**Most software is
written like an onion**



**The more layers you peel
back, the more you want to cry**

SETTER E GETTER

I campi privati possono (o non possono) essere visibili al mondo esterno

```
public class Person { 1
    private String name; 2
    private int age = -1; 3
    4
    Person(String name) { this.name = name; } 5
    6
    String getName() { return name; } 7
    8
    int getAge() { return age; }; // la visibilità di default è public 9
} 10
```

Non possiamo fare `samAge = sam.age`; ma possiamo fare `samAge = sam.getAge()`;

SETTER E GETTER (CONT.)

Per ogni campo privato, fornire un metodo setter e un getter

// non obbligatorio

```
public class Person {                                1
    private int age = -1;                             2

    // Setter                                           3
    void setAge(int age) { // aggiornamento controllato dell'età 4
        if (age > 0 && age <= 122) this.age = age;      5
        else System.out.println("Invalid age.");      6
    }                                                  7
                                                    8
    // Getter                                           9
    int getAge() { return age; } // rende l'età disponibile all'esterno 10
}                                                    11
                                                    12
```

Si possono avere più setter (overloading)

VANTAGGI DI SETTER E GETTER

I setter consentono di eseguire controlli prima di modificare i campi

- ▶ Regole di validazione, autorizzazione, ecc.

I getter consentono di modificare la rappresentazione interna senza impattare il resto del codice

- ▶ Conversione di tipo per mantenere l'interfaccia invariata

```
public class Person {  
    private short age = -1; // più efficiente in memoria  
  
    // getter retrocompatibile  
    int getAge() { return age; } // conversione di tipo implicita  
}
```

1
2
3
4
5
6

ACCESSO AI METODI

Possiamo applicare i modificatori di accesso anche ai metodi

- Di default, tutti i metodi sono **public**

```
public class Person {  
    private String name;  
    private int age = -1;  
  
    public Person(String name) { setName(name); }  
  
    // Setter  
    private void setName(name) { this.name = name; } // diversamente dall'età, name può  
    public void setAge(int age) { // essere impostato solo all'istanziamento  
        if (age > 0 && age <= 122) this.age = age;  
        else System.out.println("Invalid age.");  
    }  
  
    // Getter  
    public String getName() { return name; }  
    public int getAge() { return age; }  
}
```

MODIFICATORI DI ACCESSO

Un campo o metodo **public**

- ▶ Può essere accessibile dai metodi della stessa classe
- ▶ Può essere accessibile dai metodi di altre classi

Un campo o metodo **private**

- ▶ Può essere accessibile dai metodi della stessa classe
- ▶ Non può essere accessibile dai metodi di altre classi

MODIFICATORI DI ACCESSO



TESTING

Le funzionalità implementate dovrebbero essere testate

- ▶ Java fornisce la parola chiave `assert` per verificare condizioni a runtime
- ▶ Introdotta per verificare che alcune condizioni di funzionamento siano soddisfatte
- ▶ Le asserzioni devono essere abilitate nella JVM con `java -ea`

```
if (person == null) { ... } // gestisce l'errore
else { ... } // prosegue l'esecuzione nominale

// sostituzione più concisa
assert person != null;
// esecuzione nominale
```

1
2
3
4
5
6

TESTING (CONT.)

Le funzionalità implementate dovrebbero essere testate

- Vedremo tecniche di testing più avanzate più avanti nel corso

```
public class TestPerson { // classe di test
    private static Person person = new Person("Sam");

    private static void setAgeTest() { // metodo di test
        person.setAge(0);
        assert person.getAge() != 0 : "Age_cannot_be_0"; // messaggio di errore
    }
    // eseguire con opzione JVM -ea
    public static void main(String[] args) { setAgeTest(); }
}
```



DOMANDE E RISPOSTE

